



Alexander Ore

11/10/2025

JavaScript Programming Fundamentals

Lesson 2 – Module 1



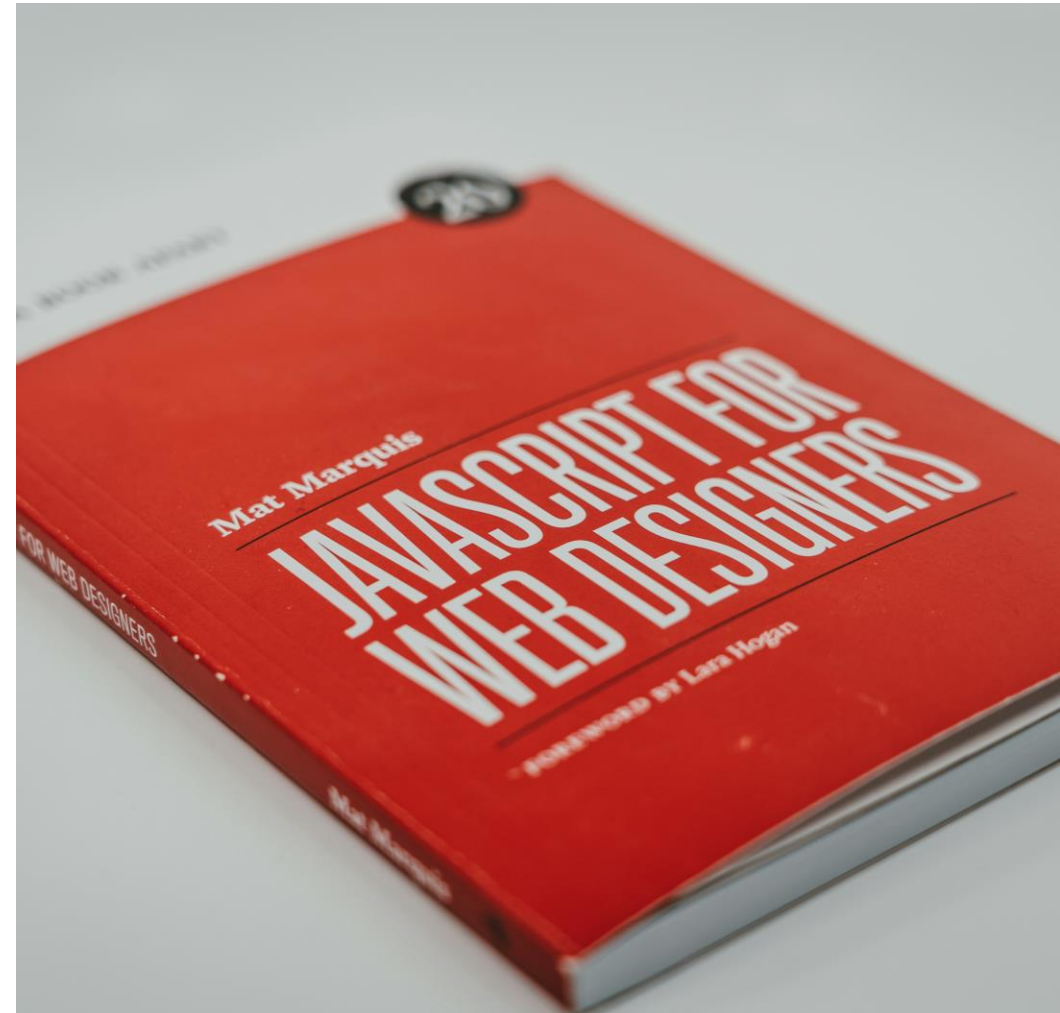


Introduction to JavaScript



What is JavaScript?

- A high-level, interpreted programming language
- Primarily used for web development (client-side and server-side)
- Dynamically typed language
- Supports multiple programming paradigms (procedural, object-oriented, functional)





“Hello, World” in JavaScript

You can run JavaScript in:

1. Web browsers (Developer Tools → Console)
2. Node.js environment
3. Online editors (CodePen, JSFiddle, Repl.it)

```
console.log("Hello, World!"); // Basic output
```

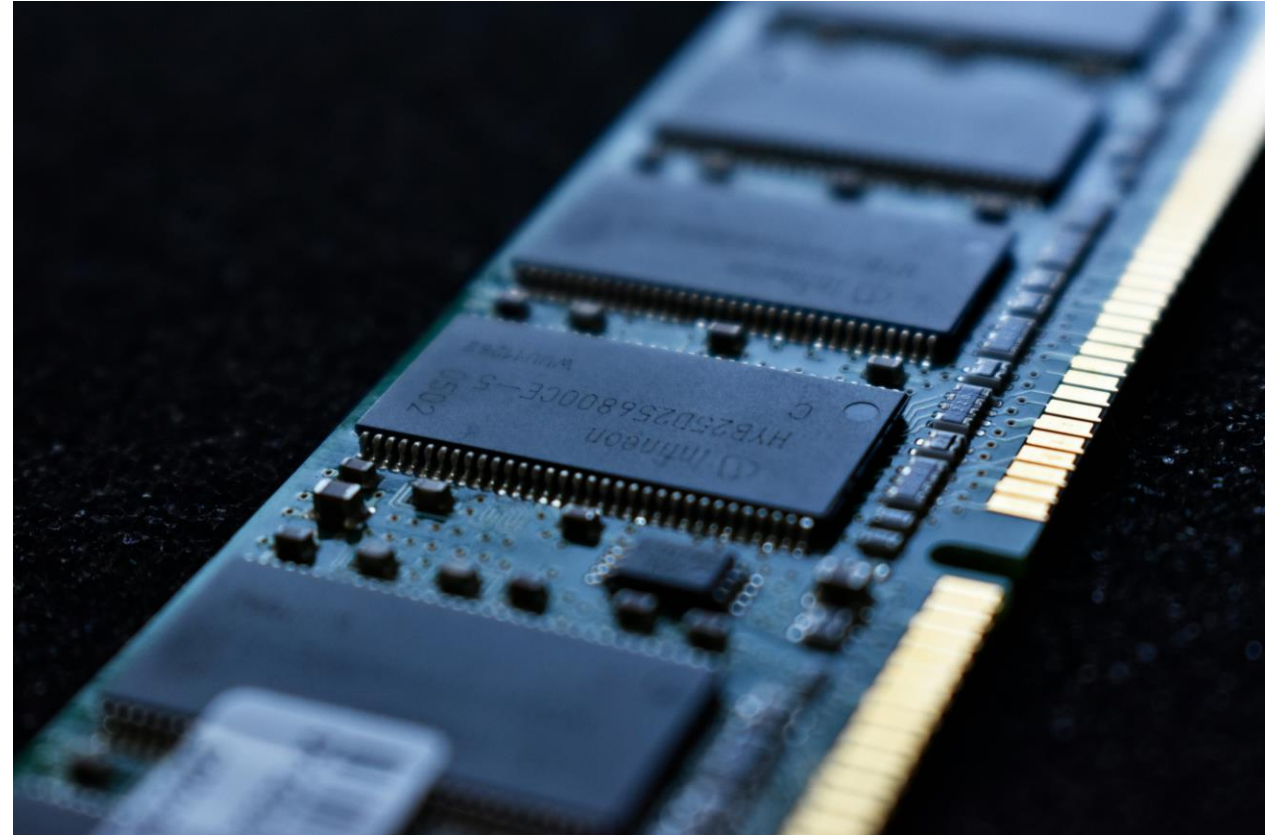


Variables



What are Variables?

- Containers for storing data values
- Named storage locations in memory





Variable Declaration

JavaScript has three ways to declare variables

```
// 1. var (function-scoped, older way)  
var firstName = "John";
```

```
// 2. let (block-scoped, modern)  
let age = 25;
```

```
// 3. const (block-scoped, cannot be reassigned)  
const PI = 3.14159;
```



Variable Naming Rules

- The camelCase is the general convention for naming variables in JavaScript.
- Use descriptive to enhance readability

```
// Valid variable names
let userName;
let user_age;
let $price;
let _temp;
let camelCase;
let user2;

// Invalid variable names
// let 2user;          // Cannot start with number
// let user-name;      // Hyphens not allowed
// let let;            // Reserved keyword
```




Variable Assignment and Reassignment

Var and Let variables may be re-assigned

Constants (const) cannot be re-assigned. The interpreter will throw an error if attempted

```
// Declaration and assignment
let score = 100;

// Reassignment
score = 200;

// Multiple variables
let x = 5, y = 10, z = 15;

// const cannot be reassigned
const MAX_SIZE = 100;
// MAX_SIZE = 200; // This would cause an error
```



Data Types



Primitive Data Types (**Number**)

Primitive Data Types are also called Native Data Types. They are also known as built-in data types because they are directly derived from the base or ancestral programming language.

```
let integer = 42;
let float = 3.14;
let negative = -15;
let scientific = 2.5e4; // 25000

// Special numeric values
let infinity = Infinity;
let notANumber = NaN;
```



Primitive Data Types (**String**)

Strings are a collection of characters. The implementation of String type differs between programming languages, but all have similar functions and usage

```
let singleQuote = 'Hello';
let doubleQuote = "World";
let backticks = `Template literal`;

// String concatenation
let fullName = "John" + " " + "Doe";

// Template literals (ES6)
let greeting = `Hello, ${fullName}! You are ${age} years old.`;

// String methods
let text = "JavaScript";
console.log(text.length);           // 10
console.log(text.toUpperCase());    // "JAVASCRIPT"
console.log(text.charAt(0));        // "J"
```



Primitive Data Types (**Boolean**)

- Boolean data types holds one of two finite values, True or False, 0 or 1

```
let isActive = true;  
let isComplete = false;
```



Primitive Data Types (**Undefined**)

- This data type is unique to JavaScript. It is returned when the data type referred cannot be inferred. Usually, it means result of computation is not available

```
let undefinedVar;  
console.log(undefinedVar); // undefined  
  
let empty = undefined;
```



Primitive Data Types (Null)

- When the value of a variable should be determined programmatically later in the run but needs to be referenced earlier, we set the value to null.
- JavaScript may return null if the resource being requested is not found

```
let emptyValue = null;
```



Complex Data Types (**Object**)

Complex data types are user defined. You may define it within your code or consume it from a library

An object is a key-value pair collection of data

```
// Object literal
let person = {
  firstName: "John",
  lastName: "Doe",
  age: 30,
  isStudent: false
};

// Accessing properties
console.log(person.firstName);    // Dot notation
console.log(person["lastName"]); // Bracket notation
```




Complex Data Types (Array)

Arrays in programming are a collection related items but in JavaScript and array can hold any object.

```
let numbers = [1, 2, 3, 4, 5];  
let mixed = [1, "hello", true, null];  
  
// Accessing elements  
console.log(numbers[0]); // 1  
console.log(numbers.length); // 5
```



Handling Types



Type Checking

Sometimes we want to determine if the data type we need for a task is what we are getting.

Another usage for type checking is in control flows

```
// typeof operator
console.log(typeof 42);           // "number"
console.log(typeof "hello");      // "string"
console.log(typeof true);         // "boolean"
console.log(typeof undefined);    // "undefined"
console.log(typeof null);         // "object" (historical bug)
console.log(typeof []);           // "object"
console.log(typeof {});           // "object"

// Better type checking
console.log(Array.isArray([]));   // true
```



Type Conversion

Sometimes we need to convert data from one type to another for use in our computations.

JavaScript defines two ways for doing this.

Explicit and **Implicit**

```
// Explicit conversion
let stringToNumber = Number("123");
let numberToString = String(123);
let toBoolean = Boolean(1); // true

// Implicit conversion (type coercion)
let result = "5" + 1; // "51" (string concatenation)
let result2 = "5" - 1; // 4 (numeric subtraction)

// Truthy and Falsy values
console.log(Boolean("")); // false
console.log(Boolean(0)); // false
console.log(Boolean(null)); // false
console.log(Boolean(undefined)); // false
console.log(Boolean(NaN)); // false
console.log(Boolean("hello")); // true
console.log(Boolean(1)); // true
console.log(Boolean([])); // true
```



Arithmetic Operators

Just like every other programming language, JavaScript out of the box is a calculator.

Special increment and decrement operators (post-pend and pre-pend) are very useful within loops and conditional statements

```
let a = 10, b = 3;

console.log(a + b); // 13 (addition)
console.log(a - b); // 7 (subtraction)
console.log(a * b); // 30 (multiplication)
console.log(a / b); // 3.333... (division)
console.log(a % b); // 1 (modulus/remainder)
console.log(a ** b); // 1000 (exponentiation)

// Increment/Decrement
let count = 5;
count++; // 6 (post-increment)
++count; // 7 (pre-increment)
count--; // 6 (post-decrement)
--count; // 5 (pre-decrement)
```



Assignment Operators

JavaScript provides several assignment plus arithmetic operators that reduces code size and shortens code

```
let x = 10;
```

```
x += 5; // x = x + 5 → 15
```

```
x -= 3; // x = x - 3 → 12
```

```
x *= 2; // x = x * 2 → 24
```

```
x /= 4; // x = x / 4 → 6
```

```
x %= 4; // x = x % 4 → 2
```

```
x **= 3; // x = x ** 3 → 8
```



Comparison Operators

JavaScript provides several assignment plus arithmetic operators that reduces code size and shortens code

```
let num1 = 5, num2 = 10, str = "5";

console.log(num1 == str);    // true (loose equality)
console.log(num1 === str);   // false (strict equality)
console.log(num1 != str);    // false (loose inequality)
console.log(num1 !== str);   // true (strict inequality)

console.log(num1 < num2);    // true
console.log(num1 > num2);    // false
console.log(num1 <= num2);   // true
console.log(num1 >= num2);   // false
```



Logical Operators

Logical operators are used in conditional blocks and loops. Logical operators resolve to either **True** or **False**

```
let isLoggedIn = true;
let hasPermission = false;

// AND (&&) - both must be true
console.log(isLoggedIn && hasPermission); // false

// OR (||) - at least one must be true
console.log(isLoggedIn || hasPermission); // true

// NOT (!) - inverts the boolean
console.log(!isLoggedIn); // false

// Short-circuit evaluation
let username = "";
let displayName = username || "Guest";
console.log(displayName); // "Guest"
```




Operator Precedence

Operator precedence in programming refers to the order in which a nested operator will be executed. Think BODMAS or PEMDAS. Operator precedence is the BODMAS of computer operations

```
// Higher precedence: () → ** → * / % → + - → comparison → logical
let result = 5 + 3 * 2;    // 11, not 16
let result2 = (5 + 3) * 2; // 16

// Use parentheses for clarity
let complex = (a + b) * c - d / e;
```



Making Decisions



Conditional Statements

if/else/else if

Conditional Statements are used to make decisions in programming. The conditional operations of the conditional block evaluates to **True** or **False**

```
let temperature = 25;

if (temperature > 30) {
    console.log("It's hot outside");
} else if (temperature > 20) {
    console.log("It's warm outside");
} else if (temperature > 10) {
    console.log("It's cool outside");
} else {
    console.log("It's cold outside");
}
```



Ternary Operator

A ternary operator is a single-liner shorter alternative for and if-else statement. For simple conditional statements this is ideal. However, for complex nested conditional statements, it is better to stick with the traditional if-else block or use a switch statement

```
let age = 20;
let canVote = age >= 18 ? "Yes" : "No";
console.log(canVote); // "Yes"

// Equivalent to:
if (age >= 18) {
    canVote = "Yes";
} else {
    canVote = "No";
}
```



Switch Statement

Instead of using a long if-else block, a **Switch** statement is a better alternative

```
let day = "Monday";

switch (day) {
  case "Monday":
    console.log("Start of work week");
    break;
  case "Friday":
    console.log("Weekend is coming!");
    break;
  case "Saturday":
  case "Sunday":
    console.log("Weekend!");
    break;
  default:
    console.log("Regular work day");
}
```



Repeating Events



For Loop

When you know the number of times you want an action repeated, then a for loop is the go-to

```
// Basic for loop
for (let i = 0; i < 5; i++) {
    console.log("Iteration: " + i);
}

// Looping through arrays
let fruits = ["apple", "banana", "orange"];
for (let i = 0; i < fruits.length; i++) {
    console.log(fruits[i]);
}
```



While Loop

When the number of repetitions cannot be determined ahead of time, the while loop is a perfect option

```
let count = 0;

while (count < 5) {
    console.log("Count: " + count);
    count++;
}

// Do-while loop (executes at least once)
let number;
do {
    number = Math.random();
    console.log("Random number: " + number);
} while (number < 0.5);
```




For...of Loop

The for...of loop is an elegant a better efficient way to loop through an array. It is the more modern method too

```
let colors = ["red", "green", "blue"];

for (let color of colors) {
  console.log(color);
}
```



For...in Loop

The for...in loop is a modern, elegant, efficient way to loop to a JavaScript object

```
let person = {  
  name: "Alice",  
  age: 25,  
  occupation: "developer"  
};  
  
for (let key in person) {  
  console.log(key + ": " + person[key]);  
}
```



Loop Control Statement

break & continue

Break and continue are useful JavaScript keywords for controlling loop execution. **continue** is used to skip an execution cycle while **break** is a form of circuit breaker for a loop

```
// break - exit loop entirely
for (let i = 0; i < 10; i++) {
  if (i === 5) {
    break; // Exit loop when i is 5
  }
  console.log(i); // 0, 1, 2, 3, 4
}

// continue - skip current iteration
for (let i = 0; i < 5; i++) {
  if (i === 2) {
    continue; // Skip when i is 2
  }
  console.log(i); // 0, 1, 3, 4
}
```



Error Handling

The for...in loop is a modern, elegant, efficient way to loop to a JavaScript object

```
try {  
    // Code that might throw an error  
    let result = riskyOperation();  
    console.log(result);  
} catch (error) {  
    // Handle the error  
    console.log("An error occurred: " + error.message);  
} finally {  
    // This always executes  
    console.log("Operation complete");  
}
```